

# Fix Tic Tac Toe Logic

Generated 9/30/2026, 12:53:13 PM • 12 messages • Source: Shared ChatGPT link  
<https://chatgpt.com/share/6abcb2e2-a2ec-83ee-8363-eac2ccad7b19>

USER

hi i have to do my se lab "Lab 4: VibeCoding Objective Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account. THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!! Duration 60 minutes Software Requirements %Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc) Instructions:

1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one

by one.

8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo.
10. Do not raise PR to the main repo (SETAPESU26)

Deliverables: Upload the following to your Lab repo under Lab-4 folder, a. Videos (before and after) b. Updated Code c. Chat history exported as a doc/pdf" these are the requirements and deliverables

"[https://github.com/SETAPESU26/09\\_tic\\_tac\\_toe](https://github.com/SETAPESU26/09_tic_tac_toe)"this the repo link i want to beging with task 1 i am done with the prev things "please give me the code "Task 1 requirements:

1. Fix win detection so that all 8 winning lines are detected, including both diagonals.
2. If the final move creates a winning line, it must be counted as a win, not a draw.
3. Once a player wins or the game becomes a draw, no further moves should be accepted." """"

ai: a simple computer opponent for the second player (O).

Picks a random empty cell - this is intentionally simple. The point

of this lab is testing your own win/draw/turn logic against a working

opponent, not building a hard-to-beat AI.

""""

```
import random
```

```
def choose_move(board):
```

```
    empty_cells = [
```

```
        (r, c) for r in range(3) for c in range(3) if board[r][c] is None
```

```
    ]
```

```
    if not empty_cells:
```

```
        return None
```

```
    return random.choice(empty_cells)"""" GameEngine: owns the board, turn state, and round-end logic.
```

You (the player) always play X and click to move. The computer always plays O and moves automatically right after you, using a simple random-move AI (see game/ai.py) - this is given infrastructure, not something you need to build.

Starter version: no scoreboard yet, no first-player choice, and only one combined reset control. Win/draw detection has known bugs (see game/rules.py and check\_round\_end below) that Task 1 asks you to fix, and move validation has a known gap (see handle\_click) that Task 3 asks you to fix. """

```
from game.rules import check_winner, is_board_full from game.renderer import board_pos_to_cell from game.ai
import choose_move

HUMAN_SYMBOL = 'X' COMPUTER_SYMBOL = 'O'

class GameEngine:
    def __init__(self):
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = 'X'
        self.round_over = False
        self.winner = None # 'X', 'O', or None (meaning draw, only valid when round_over)

    def handle_click(self, pos):
        if self.current_player != HUMAN_SYMBOL:
            return # not your turn - the computer is about to move (or already has)
        cell = board_pos_to_cell(pos)
        if cell is None:
            return row, col = cell
        self.board[row][col] = self.current_player # BUG: doesn't check if the cell is already occupied
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'
        self._maybe_take_computer_turn()

    def _maybe_take_computer_turn(self):
        if self.round_over or self.current_player != COMPUTER_SYMBOL:
            return
        move = choose_move(self.board)
        if move is None:
            return row, col = move
        self.board[row][col] = self.current_player
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'

    def handle_keydown(self, key):
        import pygame
        if key == pygame.K_r:
            self.__init__()

    def check_round_end(self):
        if is_board_full(self.board): # BUG: checked before looking for a winner
            self.round_over = True
            self.winner = None
            return winner = check_winner(self.board)
        if winner:
            self.round_over = True
            self.winner = winner

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_board(surface, self.board)
        turn_label = "Your turn (X)" if self.current_player == HUMAN_SYMBOL else "Computer's turn (O)"
        renderer.draw_text(surface, font, turn_label, (10, 20))

    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"
        renderer.draw_banner(surface, font, f"{text} Press R for a new round.")
        """ renderer: all pygame drawing lives here, kept separate from game logic. """

import pygame

WIDTH, HEIGHT = 400, 500
BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)

def board_pos_to_cell(pos):
    x, y = pos
    y -= BOARD_TOP
    if not (0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE):
        return None
    col = x // CELL_SIZE
    row = y // CELL_SIZE
    return int(row), int(col)

def draw_board(surface, board):
    surface.fill(COLOR_BG)
    for i in range(1, 3):
        pygame.draw.line(surface, COLOR_LINE, (i * CELL_SIZE, BOARD_TOP), (i * CELL_SIZE, BOARD_TOP + BOARD_SIZE), 3)
    pygame.draw.line(surface, COLOR_LINE, (0, BOARD_TOP + i * CELL_SIZE), (BOARD_SIZE, BOARD_TOP + i * CELL_SIZE), 3)

    for r in range(3):
        for c in range(3):
            symbol = board[r][c]
            if symbol is None:
                continue
            center = (c * CELL_SIZE + CELL_SIZE // 2, BOARD_TOP + r * CELL_SIZE + CELL_SIZE // 2)
            if symbol == 'X':
                offset = CELL_SIZE // 3
                pygame.draw.line(surface, COLOR_X, (center[0]-offset, center[1]-offset), (center[0]+offset, center[1]+offset), 6)
                pygame.draw.line(surface, COLOR_X, (center[0]+offset, center[1]-offset), (center[0]-offset, center[1]+offset), 6)
            else:
                pygame.draw.circle(surface, COLOR_O, center, CELL_SIZE // 3, 6)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect =
```

```
surf.get_rect(center=(surface.get_width() // 2, BOARD_TOP + BOARD_SIZE + 40)) surface.blit(surf, rect) """ rules: win/draw detection for a 3x3 board.
```

```
board is a 3x3 list of lists, each cell is None, 'X', or 'O'. """
```

```
def check_winner(board): lines = [] for r in range(3): lines.append([board[r][0], board[r][1], board[r][2]]) for c in range(3): lines.append([board[0][c], board[1][c], board[2][c]]) # NOTE: diagonals are not included here
```

```
for line in lines: if line[0] is not None and line[0] == line[1] == line[2]: return line[0] return None
```

```
def is_board_full(board): return all(cell is not None for row in board for cell in row) " """ Tic-Tac-Toe(Lab Starter)
```

Run with: `python3 main.py`

Click a cell to place your symbol. """

```
import pygame
```

```
from game.game_engine import GameEngine from game.renderer import WINDOW_SIZE
```

```
def main(): pygame.init() screen = pygame.display.set_mode(WINDOW_SIZE)
```

```
pygame.display.set_caption("Tic-Tac-Toe") clock = pygame.time.Clock() font = pygame.font.SysFont("consolas", 22)
```

```
engine = GameEngine() running = True while running: for event in pygame.event.get(): if event.type ==
```

```
pygame.QUIT: running = False elif event.type == pygame.MOUSEBUTTONDOWN: engine.handle_click(event.pos)
```

```
elif event.type == pygame.KEYDOWN: engine.handle_keydown(event.key)
```

```
engine.draw(screen, font)
```

```
pygame.display.flip() clock.tick(60)
```

```
pygame.quit()
```

```
if name == "main": main() """
```

#### ASSISTANT

Yes— for **Task 1**, you only need to modify the game logic/rules. The AI, renderer, and `main.py` can stay as they are.

There are **three things to fix**:

1. Add both diagonals to win detection.
2. Check for a winner **before** checking whether the board is full.
3. Prevent any moves after the round has ended.

#### 1. Replace `game/rules.py` with this

PYTHON

```
"""
```

```
rules: win/draw detection for a 3x3 board.
```

```
board is a 3x3 list of lists, each cell is None, 'X', or 'O'.  
"""
```

```
def check_winner(board):
```

```
    # All 8 possible winning lines:
```

```
    # 3 rows + 3 columns + 2 diagonals
```

```
    lines = [
```

```
        # Rows
```

```
        [board[0][0], board[0][1], board[0][2]],
```

```
        [board[1][0], board[1][1], board[1][2]],
```

```
        [board[2][0], board[2][1], board[2][2]],
```

```

    # Columns
    [board[0][0], board[1][0], board[2][0]],
    [board[0][1], board[1][1], board[2][1]],
    [board[0][2], board[1][2], board[2][2]],

    # Diagonals
    [board[0][0], board[1][1], board[2][2]],
    [board[0][2], board[1][1], board[2][0]],
]

for line in lines:
    if line[0] is not None and line[0] == line[1] == line[2]:
        return line[0]

return None

def is_board_full(board):
    return all(cell is not None for row in board for cell in row)

```

## 2. Replace `game/game_engine.py` with this

```

PYTHON
"""
GameEngine: owns the board, turn state, and round-end logic.

You (the player) always play X and click to move.
The computer always plays O and moves automatically right after you,
using a simple random-move AI.
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'

class GameEngine:
    def __init__(self):
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = HUMAN_SYMBOL
        self.round_over = False
        self.winner = None    # 'X', 'O', or None (draw)

    def handle_click(self, pos):
        # Do not accept moves after the round is over
        if self.round_over:
            return

        # Only allow the human player to make a move
        if self.current_player != HUMAN_SYMBOL:
            return

        cell = board_pos_to_cell(pos)

        if cell is None:
            return

        row, col = cell

        # Do not allow a move in an occupied cell

```

```

        if self.board[row][col] is not None:
            return

        # Place the human player's move
        self.board[row][col] = self.current_player

        # Check if the move ended the round
        self.check_round_end()

        # If the round ended, do not switch turns or let the computer move
        if self.round_over:
            return

        # Switch to computer's turn
        self.current_player = COMPUTER_SYMBOL

        self._maybe_take_computer_turn()

def _maybe_take_computer_turn(self):
    # Computer should not move if the round is already over
    if self.round_over or self.current_player != COMPUTER_SYMBOL:
        return

    move = choose_move(self.board)

    if move is None:
        return

    row, col = move

    # Place the computer's move
    self.board[row][col] = self.current_player

    # Check if the computer's move ended the round
    self.check_round_end()

    # If the round ended, don't switch turns
    if self.round_over:
        return

    # Switch back to human's turn
    self.current_player = HUMAN_SYMBOL

def handle_keydown(self, key):
    import pygame

    if key == pygame.K_r:
        self.__init__()

def check_round_end(self):
    # IMPORTANT:
    # Check winner FIRST.
    # This ensures a winning final move is counted as a win
    # instead of incorrectly being counted as a draw.

    winner = check_winner(self.board)

    if winner:
        self.round_over = True
        self.winner = winner
        return

    # If nobody has won and the board is full, it is a draw
    if is_board_full(self.board):
        self.round_over = True
        self.winner = None

```

```

        return

def draw(self, surface, font):
    from game import renderer

    renderer.draw_board(surface, self.board)

    turn_label = (
        "Your turn (X)"
        if self.current_player == HUMAN_SYMBOL
        else "Computer's turn (O)"
    )

    renderer.draw_text(surface, font, turn_label, (10, 20))

    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"
        renderer.draw_banner(
            surface,
            font,
            f"{text} Press R for a new round."
        )

```

### What exactly was fixed?

| Requirement                             | Fix                                                            |
|-----------------------------------------|----------------------------------------------------------------|
| All 8 winning lines                     | Added 2 diagonals in check_winner()                            |
| Winning final move should be a win      | check_round_end() checks check_winner() before is_board_full() |
| No moves after win/draw                 | handle_click() immediately returns if self.round_over          |
| No accidental overwrite                 | Added occupied-cell check                                      |
| Computer shouldn't move after game ends | Added if self.round_over: return after checking its move       |

The **occupied-cell check** is technically a small extra safety fix related to move validation, so if your teacher expects Task 3 to specifically address that, you can mention that Task 1's round-end protection is implemented here while the dedicated Task 3 requirements may add further validation.

### For your Vibe Coding requirement

Since your lab specifically says **"Write prompts and fix the broken code"**, don't just paste the final code into your submission. You should keep the ChatGPT conversation showing the prompt you used.

A good prompt for Task 1 would be:

:::writing{variant="standard" id="48317" title="Task 1 Vibe Coding Prompt"} I have a Python/Pygame Tic-Tac-Toe game. Fix the existing code according to these requirements without changing the overall architecture or adding unnecessary features:

1. Fix win detection so that all 8 winning lines are detected: 3 rows, 3 columns, and both diagonals.
2. If the final move creates a winning line, it must be counted as a win, not a draw. Therefore, winner detection must happen before board-full/draw detection.
3. Once a player wins or the game becomes a draw, no further moves should be accepted, and the computer should not make another move.
4. Preserve the existing X-human/O-computer gameplay and random AI.
5. Show me exactly which files need to be changed and provide the complete corrected code for those files.

...

Then run:

```
BASH
python3 main.py
```

## How to test Task 1

You should specifically test:

### Test 1 — Row win

```
TEXT
X | X | X
-----
O | O | 
-----
  |  | 
```

### Test 2 — Column win

```
TEXT
X | O | 
-----
X | O | 
-----
X |  | 
```

### Test 3 — Main diagonal

```
TEXT
X | O | 
-----
O | X | 
-----
  |  | X
```

### Test 4 — Other diagonal

```
TEXT
O | X | 
-----
X | O | 
-----
  |  | X
```

### Test 5 — Full board where the final move wins

This one is particularly important because the original code had:

```
PYTHON
if is_board_full(self.board):
```

**before:**

PYTHON

```
winner = check_winner(self.board)
```

So the final winning move could incorrectly display **"Draw!"**.

Your corrected order is:

PYTHON

```
winner = check_winner(self.board)

if winner:
    self.round_over = True
    self.winner = winner
    return

if is_board_full(self.board):
    self.round_over = True
    self.winner = None
```

That directly satisfies requirement #2.

After you're satisfied with Task 1, **record your evidence/video as required by the lab and save the chat history** before moving to the next task.

Do you want to do **Task 2 next**, or should we first go through **Task 1 testing + Git commit/push**?

USER

i am done with task 1 implementation, testing and pushing, now for task 2 "Task 2 requirements: Track the number of rounds won by X and O. Track the number of draws and display the scoreboard visibly in the game. When a new round is started, the board and round state should reset, but the scoreboard must remain unchanged. The scoreboard should only be cleared when the entire match is explicitly reset. The separate match-reset control will be implemented later."

ASSISTANT

Yes. For **Task 2**, we'll build the scoreboard while preserving everything you already fixed in Task 1.

The clean approach is:

- `GameEngine` stores `x_wins`, `o_wins`, and `draws`.
- When a round ends, the appropriate score increases **exactly once**.
- Pressing **R** starts a new round but keeps the scoreboard.
- The scoreboard is drawn visibly at the top.
- We **won't implement the separate match-reset control yet**, because your requirement explicitly says that comes later.

### Step 1 — Update `game/game_engine.py`

Replace your current `game/game_engine.py` with this:

PYTHON

```
"""
GameEngine: owns the board, turn state, round-end logic,
and match scoreboard.
```

You (the player) always play X and click to move.  
The computer always plays O and moves automatically right after you,  
using a simple random-move AI.

```
"""
```

```
from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move
```

```
HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'
```

```
class GameEngine:
    def __init__(self):
        # Round state
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = HUMAN_SYMBOL
        self.round_over = False
        self.winner = None

        # Match scoreboard
        self.x_wins = 0
        self.o_wins = 0
        self.draws = 0

    def handle_click(self, pos):
        # Do not accept moves after the round is over
        if self.round_over:
            return

        # Only allow the human player to make a move
        if self.current_player != HUMAN_SYMBOL:
            return

        cell = board_pos_to_cell(pos)

        if cell is None:
            return

        row, col = cell

        # Do not allow a move in an occupied cell
        if self.board[row][col] is not None:
            return

        # Place the human player's move
        self.board[row][col] = self.current_player

        # Check if the round ended
        self.check_round_end()

        # If the round ended, do not switch turns
        if self.round_over:
            return

        # Switch to computer's turn
        self.current_player = COMPUTER_SYMBOL

        self._maybe_take_computer_turn()

    def _maybe_take_computer_turn(self):
        # Computer should not move if the round is already over
        if self.round_over or self.current_player != COMPUTER_SYMBOL:
            return
```

```

move = choose_move(self.board)

if move is None:
    return

row, col = move

# Place the computer's move
self.board[row][col] = self.current_player

# Check if the round ended
self.check_round_end()

# If the round ended, don't switch turns
if self.round_over:
    return

# Switch back to human's turn
self.current_player = HUMAN_SYMBOL

def handle_keydown(self, key):
    import pygame

    if key == pygame.K_r:
        self.reset_round()

def reset_round(self):
    """
    Reset only the current round.

    The scoreboard is intentionally preserved.
    """
    self.board = [[None] * 3 for _ in range(3)]
    self.current_player = HUMAN_SYMBOL
    self.round_over = False
    self.winner = None

def check_round_end(self):
    # Check winner first so a winning final move
    # is not incorrectly counted as a draw.
    winner = check_winner(self.board)

    if winner:
        self.round_over = True
        self.winner = winner

        # Update the scoreboard
        if winner == HUMAN_SYMBOL:
            self.x_wins += 1
        elif winner == COMPUTER_SYMBOL:
            self.o_wins += 1

    return

# If nobody has won and the board is full, it is a draw
if is_board_full(self.board):
    self.round_over = True
    self.winner = None

    # Update draw count
    self.draws += 1

def draw(self, surface, font):
    from game import renderer

```

```

    renderer.draw_board(surface, self.board)

    # Display scoreboard
    scoreboard = (
        f"X Wins: {self.x_wins}    "
        f"O Wins: {self.o_wins}    "
        f"Draws: {self.draws}"
    )

    renderer.draw_text(
        surface,
        font,
        scoreboard,
        (10, 20)
    )

    # Display current turn
    turn_label = (
        "Your turn (X)"
        if self.current_player == HUMAN_SYMBOL
        else "Computer's turn (O)"
    )

    renderer.draw_text(
        surface,
        font,
        turn_label,
        (10, 55)
    )

    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"

        renderer.draw_banner(
            surface,
            font,
            f"{text} Press R for a new round."
        )

```

## Step 2 — No changes are required to `rules.py`

Keep your Task 1 version of:

TEXT

`game/rules.py`

exactly as it is.

## Step 3 — `main.py` also stays unchanged

You don't need to modify `main.py`.

The existing:

PYTHON

```

elif event.type == pygame.KEYDOWN:
    engine.handle_keydown(event.key)

```

already allows the `R` key to reach `GameEngine`.

## What changed for Task 2?

The important addition is this:

PYTHON

```
self.x_wins = 0
self.o_wins = 0
self.draws = 0
```

These are **match-level variables**.

Then, when X wins:

PYTHON

```
if winner == HUMAN_SYMBOL:
    self.x_wins += 1
```

When O wins:

PYTHON

```
elif winner == COMPUTER_SYMBOL:
    self.o_wins += 1
```

And when there's a draw:

PYTHON

```
self.draws += 1
```

## Most important part: round reset

Instead of doing:

PYTHON

```
self.__init__()
```

when **R** is pressed, we're doing:

PYTHON

```
self.reset_round()
```

and `reset_round()` only resets:

PYTHON

```
self.board
self.current_player
self.round_over
self.winner
```

It **doesn't touch**:

PYTHON

```
self.x_wins
self.o_wins
self.draws
```

So, for example:

TEXT

```
R o u n d   1   ! '   X   w i n s

X Wins: 1      0 Wins: 0      Draws: 0

Press R

New board

X Wins: 1      0 Wins: 0      Draws: 0
```

That's exactly what Task 2 asks for.

## One thing to watch for

Because the scoreboard is drawn at:

PYTHON

```
(10, 20)
```

and the turn text is drawn at:

PYTHON

```
(10, 55)
```

your board still starts at `y = 100`, so there is enough space for both.

Your screen should look roughly like:

TEXT

```
X Wins: 1      0 Wins: 0      Draws: 0

Your turn (X)

  X   |   0   |
  ---+---+---
  X   |       |
  ---+---+---
       |   0   |
```

## Task 2 testing checklist

Before pushing, test these **four things**:

- **X wins !'** `X Wins` increases by 1.
- **O wins !'** `O Wins` increases by 1.
- **Draw !'** `Draws` increases by 1.

- Press **R** after a completed round !' board resets, but scoreboard remains unchanged.

Also make sure the score **doesn't increase repeatedly** while the winning board is sitting on screen. Each completed round should contribute exactly one point.

For your VibeCoding submission, I would use a prompt like this rather than simply saying "add scoreboard":

:::writing{variant="standard" id="73164" title="Task 2 Vibe Coding Prompt"} I have already completed Task 1 of my Python/Pygame Tic-Tac-Toelab. Keep all existing Task 1 functionality unchanged.

Now implement ONLY the following Task 2 requirements:

1. Track the number of rounds won by X.
2. Track the number of rounds won by O.
3. Track the number of draws.
4. Display the scoreboard visibly in the game.
5. When a new round is started using the existing R key, reset only the board and round state. The X wins, O wins, and draws scoreboard values must remain unchanged.
6. The scoreboard should only be cleared when an explicit match-reset feature is implemented later. Do NOT implement the separate match-reset control yet.
7. Make sure each completed round updates the scoreboard exactly once.
8. Preserve the existing random computer opponent and all Task 1 win/draw/game-over behavior.
9. Minimize changes to the existing project and tell me which files need to be changed.

Provide the complete updated code for every file that needs modification. :::

**One correction to keep in mind:** your original `__init__()` resets everything, so simply calling `self.__init__()` for **R** would erase the scoreboard. That's why Task 2 needs a separate `reset_round()` method.

After testing, commit and push Task 2 just like you did for Task 1.

USER

what changes should be done in `renderer.py`?

ASSISTANT

For the Task 2 implementation I gave you, **`renderer.py` does not strictly need any changes**. The scoreboard can be displayed using the existing `draw_text()` function from `GameEngine`.

However, there is one small improvement I recommend: add a dedicated `draw_scoreboard()` function to `renderer.py`. This keeps the rendering logic separated properly, which matches the structure of your starter project.

### Recommended `game/renderer.py`

Keep everything you already have, and add this function:

PYTHON

```
def draw_scoreboard(surface, font, x_wins, o_wins, draws):
    text = f"X Wins: {x_wins}    O Wins: {o_wins}    Draws: {draws}"
    draw_text(surface, font, text, (10, 20))
```

So your complete `renderer.py` would be:

# PYTHON

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""
```

```
import pygame
```

```
WIDTH, HEIGHT = 400, 500
BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100
WINDOW_SIZE = (WIDTH, HEIGHT)
```

```
COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)
```

```
def board_pos_to_cell(pos):
    x, y = pos
    y -= BOARD_TOP

    if not (0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE):
        return None

    col = x // CELL_SIZE
    row = y // CELL_SIZE

    return int(row), int(col)
```

```
def draw_board(surface, board):
    surface.fill(COLOR_BG)

    for i in range(1, 3):
        pygame.draw.line(
            surface,
            COLOR_LINE,
            (i * CELL_SIZE, BOARD_TOP),
            (i * CELL_SIZE, BOARD_TOP + BOARD_SIZE),
            3
        )

        pygame.draw.line(
            surface,
            COLOR_LINE,
            (0, BOARD_TOP + i * CELL_SIZE),
            (BOARD_SIZE, BOARD_TOP + i * CELL_SIZE),
            3
        )

    for r in range(3):
        for c in range(3):
            symbol = board[r][c]

            if symbol is None:
                continue

            center = (
                c * CELL_SIZE + CELL_SIZE // 2,
                BOARD_TOP + r * CELL_SIZE + CELL_SIZE // 2
            )
```

```

        if symbol == 'X':
            offset = CELL_SIZE // 3

            pygame.draw.line(
                surface,
                COLOR_X,
                (center[0] - offset, center[1] - offset),
                (center[0] + offset, center[1] + offset),
                6
            )

            pygame.draw.line(
                surface,
                COLOR_X,
                (center[0] + offset, center[1] - offset),
                (center[0] - offset, center[1] + offset),
                6
            )

        else:
            pygame.draw.circle(
                surface,
                COLOR_O,
                center,
                CELL_SIZE // 3,
                6
            )

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_scoreboard(surface, font, x_wins, o_wins, draws):
    """
    Draw the current match scoreboard.
    """
    text = f"X Wins: {x_wins}    O Wins: {o_wins}    Draws: {draws}"
    draw_text(surface, font, text, (10, 20))

def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))

    rect = surf.get_rect(
        center=(
            surface.get_width() // 2,
            BOARD_TOP + BOARD_SIZE + 40
        )
    )

    surface.blit(surf, rect)

```

### Then change `draw()` in `game_engine.py`

Instead of putting the scoreboard formatting directly inside `GameEngine`, use:

```

PYTHON
renderer.draw_scoreboard(
    surface,
    font,
    self.x_wins,
    self.o_wins,
    self.draws

```

```
)
```

So this section:

PYTHON

```
scoreboard = (  
    f"X Wins: {self.x_wins}    "  
    f"O Wins: {self.o_wins}    "  
    f"Draws: {self.draws}"  
)  
  
renderer.draw_text(  
    surface,  
    font,  
    scoreboard,  
    (10, 20)  
)
```

becomes:

PYTHON

```
renderer.draw_scoreboard(  
    surface,  
    font,  
    self.x_wins,  
    self.o_wins,  
    self.draws  
)
```

## Why I'd do it this way

Your starter README says:

**renderer: all pygame drawing lives here, kept separate from game logic.**

So this is cleaner:

`game\_engine.py` !' keeps the scores and decides **what** should be displayed.

`renderer.py` !' decides **how** the scoreboard is visually displayed.

This is also a better implementation to show for your **VibeCoding lab**, because you're making a minimal, logical modification rather than putting all new rendering code into the game engine.

**So for Task 2, modify two files:**

- `game/game_engine.py` !' scoreboard state + score updating + round reset
- `game/renderer.py` !' `draw_scoreboard()`

`main.py`, `rules.py`, and `ai.py` don't need changes.

USER

i am done with task 2, now i want the code for task3 along with the explanation of the changes that we will commit in all the files. (reserve task1 and 2) "task 3 requirements: An occupied cell cannot be overwritten. An invalid click must not change the board and not change the turn. All the valid moves must continue working."

## ASSISTANT

Absolutely. Since **Task 1 and Task 2 are already completed and should be preserved**, Task 3 should make the **minimum necessary changes**.

The good news is that most of the Task 3 logic was already introduced while fixing Task 1. We should now make sure it is explicitly and cleanly implemented without breaking your scoreboard or round-reset behavior.

## Task 3 — What we need to achieve

There are three requirements:

### 1. Occupied cell cannot be overwritten

- If a cell already contains `X` or `O`, clicking it does nothing.

### 2. Invalid click changes nothing

- The board must remain exactly the same.
- The current player/turn must remain the same.
- Examples:
  - Clicking outside the board.
  - Clicking an already occupied cell.
  - Clicking while it is the computer's turn.
  - Clicking after the round has ended.

### 3. Valid moves continue working

- Clicking an empty cell during X's turn places `X`.
- The computer then makes its valid `O` move.
- The turn changes normally.
- Win/draw detection from Task 1 continues working.
- Scoreboard from Task 2 continues working.

## Files we will change

For Task 3, I recommend changing **only**:

TEXT

```
game/game_engine.py
```

You do **not** need to modify:

TEXT

```
game/rules.py
game/renderer.py
game/ai.py
main.py
```

Your Task 1 and Task 2 implementations remain intact.

## 1. game/game\_engine.py

Replace your current `game/game_engine.py` with this version.

PYTHON

"""

GameEngine: owns the board, turn state, round-end logic,  
and match scoreboard.

You (the player) always play X and click to move.  
The computer always plays O and moves automatically right after you,  
using a simple random-move AI.

"""

```
from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move
```

```
HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'
```

```
class GameEngine:
    def __init__(self):
        # Round state
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = HUMAN_SYMBOL
        self.round_over = False
        self.winner = None

        # Match scoreboard
        self.x_wins = 0
        self.o_wins = 0
        self.draws = 0

    def handle_click(self, pos):
        # Do not accept any moves after the round is over
        if self.round_over:
            return

        # Only allow the human player to make a move
        if self.current_player != HUMAN_SYMBOL:
            return

        # Convert mouse position into a board cell
        cell = board_pos_to_cell(pos)

        # Invalid click outside the board
        if cell is None:
            return

        row, col = cell

        # Do not allow an occupied cell to be overwritten
        if self.board[row][col] is not None:
            return

        # Place the human player's move
        self.board[row][col] = HUMAN_SYMBOL

        # Check whether the move ended the round
        self.check_round_end()

        # If the round ended, do not change the turn
        # and do not let the computer move.
        if self.round_over:
            return

        # Switch to computer's turn
        self.current_player = COMPUTER_SYMBOL
```

```

        # Let the computer make its move
        self._maybe_take_computer_turn()

def _maybe_take_computer_turn(self):
    # Computer should not move if the round is over
    # or if it is not the computer's turn.
    if self.round_over or self.current_player != COMPUTER_SYMBOL:
        return

    move = choose_move(self.board)

    # No valid move available
    if move is None:
        return

    row, col = move

    # Extra safety check:
    # Never overwrite an occupied cell.
    if self.board[row][col] is not None:
        return

    # Place the computer's move
    self.board[row][col] = COMPUTER_SYMBOL

    # Check whether the computer's move ended the round
    self.check_round_end()

    # If the round ended, don't change the turn
    if self.round_over:
        return

    # Switch back to human's turn
    self.current_player = HUMAN_SYMBOL

def handle_keydown(self, key):
    import pygame

    if key == pygame.K_r:
        self.reset_round()

def reset_round(self):
    """
    Reset only the current round.

    The match scoreboard is preserved.
    """
    self.board = [[None] * 3 for _ in range(3)]
    self.current_player = HUMAN_SYMBOL
    self.round_over = False
    self.winner = None

def check_round_end(self):
    # Check winner FIRST.
    # This ensures that a winning final move
    # is counted as a win instead of a draw.

    winner = check_winner(self.board)

    if winner:
        self.round_over = True
        self.winner = winner

        # Update the scoreboard
        if winner == HUMAN_SYMBOL:

```

```

        self.x_wins += 1
    elif winner == COMPUTER_SYMBOL:
        self.o_wins += 1

    return

# If nobody has won and the board is full,
# the round is a draw.
if is_board_full(self.board):
    self.round_over = True
    self.winner = None
    self.draws += 1

def draw(self, surface, font):
    from game import renderer

    renderer.draw_board(surface, self.board)

    # Display scoreboard
    renderer.draw_scoreboard(
        surface,
        font,
        self.x_wins,
        self.o_wins,
        self.draws
    )

    # Display current turn
    turn_label = (
        "Your turn (X)"
        if self.current_player == HUMAN_SYMBOL
        else "Computer's turn (O)"
    )

    renderer.draw_text(
        surface,
        font,
        turn_label,
        (10, 55)
    )

    # Display round result
    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"

        renderer.draw_banner(
            surface,
            font,
            f"{text} Press R for a new round."
        )

```

## What changed from Task 2?

There are **four important Task 3 changes**.

### Change 1 — Check whether the cell is occupied

This is the most important part:

PYTHON

```

if self.board[row][col] is not None:
    return

```

Suppose the board is:

TEXT

```
X | O |  
-----  
  | X |  
-----  
  |  |
```

If you click the top-left cell again:

TEXT

```
X
```

already exists there.

The function simply returns.

Therefore:

- `X` isn't overwritten.
- The board doesn't change.
- The turn doesn't change.
- The computer isn't triggered.

## Change 2 — Invalid clicks outside the board

We already have:

PYTHON

```
cell = board_pos_to_cell(pos)  
  
if cell is None:  
    return
```

`board_pos_to_cell()` returns `None` when the click isn't inside the 3×3 board.

So something like clicking here:

TEXT

```
+-----+  
| scoreboard |  
|           |  
|     BOARD  |  
|           |  
|           |  
+-----+
```

doesn't modify anything.

Most importantly, the code returns **before**:

PYTHON

```
self.board[row][col] = HUMAN_SYMBOL
```

and before:

PYTHON

```
self.current_player = COMPUTER_SYMBOL
```

So the turn remains unchanged.

### Change 3 — Computer move gets an extra safety check

We also add:

PYTHON

```
if self.board[row][col] is not None:
    return
```

inside `_maybe_take_computer_turn()`.

Your `ai.py` already chooses from empty cells, so normally this shouldn't be necessary.

But it protects the game engine from accidentally overwriting a cell if the AI ever returns an invalid/occupied position.

It also makes the **"occupied cells can never be overwritten"** rule true for both players.

### Change 4 — Turn changes only after a valid move

Look carefully at the order:

PYTHON

```
if cell is None:
    return

row, col = cell

if self.board[row][col] is not None:
    return

self.board[row][col] = HUMAN_SYMBOL
```

Only **after** those checks do we eventually reach:

PYTHON

```
self.current_player = COMPUTER_SYMBOL
```

Therefore:

#### Invalid click

TEXT

```
click
!"
invalid?
!"
return
```

Nothing changes.

#### Valid click

TEXT

```
click
!"
valid cell?
!" y e s
empty?
!" y e s
place X
!"
check win/draw
!"
switch to 0
!"
computer moves
```

This is exactly the behavior your Task 3 requires.

## What stays from Task 1 and Task 2?

This is important for your lab because you're supposed to **build on the same code**.

### Task 1 remains

Your `rules.py` still has all 8 winning lines:

PYTHON

```
# Rows
[board[0][0], board[0][1], board[0][2]]
[board[1][0], board[1][1], board[1][2]]
[board[2][0], board[2][1], board[2][2]]

# Columns
[board[0][0], board[1][0], board[2][0]]
[board[0][1], board[1][1], board[2][1]]
[board[0][2], board[1][2], board[2][2]]

# Diagonals
[board[0][0], board[1][1], board[2][2]]
[board[0][2], board[1][1], board[2][0]]
```

And winner detection still happens before draw detection.

### Task 2 remains

The scoreboard still contains:

PYTHON

```
self.x_wins = 0
self.o_wins = 0
self.draws = 0
```

And pressing `R` still calls:

PYTHON

```
self.reset_round()
```

rather than:

PYTHON

```
self.__init__()
```

So your scoreboard **doesn't get erased** when starting a new round.

Your `renderer.py` also stays exactly as you finished it in Task 2, including:

PYTHON

```
def draw_scoreboard(surface, font, x_wins, o_wins, draws):  
    text = f"X Wins: {x_wins}    O Wins: {o_wins}    Draws: {draws}"  
    draw_text(surface, font, text, (10, 20))
```

---

## Task 3 testing

Before pushing, I recommend doing these tests.

### Test 1 — Occupied X cell

Place X somewhere.

Then click the same cell again.

Expected:

TEXT

```
Board: unchanged  
Turn: unchanged  
Score: unchanged
```

---

### Test 2 — Occupied O cell

Allow the computer to place O.

Click the O cell.

Expected:

TEXT

```
O remains O  
No X appears there  
No additional O move is triggered
```

---

### Test 3 — Outside the board

Click somewhere above the board, for example on the scoreboard.

Expected:

TEXT

```
Board: unchanged  
Turn: unchanged
```

This is a particularly good test because your scoreboard occupies the area above the board.

---

### Test 4 — Valid empty-cell click

Click an empty cell.

Expected:

TEXT

```
X is placed  
!' computer automatically places O  
!' turn returns to X
```

---

### Test 5 — Invalid click after game ends

Finish a round.

Then click any cell.

Expected:

TEXT

```
Board: unchanged  
Score: unchanged
```

This confirms your Task 1 game-over protection still works.

---

## Your Vibe Coding prompt for Task 3

Since your lab specifically wants you to demonstrate prompting, I'd use a prompt that tells the AI to **preserve Tasks 1 and 2**:

```
:::writing{variant="standard" id="61582" title="Task 3 Vibe Coding Prompt"} I have already completed Task 1 and Task 2 of my Python/Pygame Tic-Tac-Toelab. Do not remove or break any existing functionality.
```

Task 1 functionality that must remain:

- All 8 winning lines including both diagonals are detected.
- A winning final move is counted as a win instead of a draw.
- No moves are accepted after a round ends.

Task 2 functionality that must remain:

- Track X wins, O wins, and draws.
- Display the scoreboard visibly.
- Pressing R starts a new round while preserving the scoreboard.
- The separate match-reset control must NOT be implemented yet.

Now implement Task 3:

1. An occupied cell cannot be overwritten.
2. An invalid click must not change the board.
3. An invalid click must not change the current turn.
4. Valid moves must continue working normally.
5. Invalid clicks include clicking outside the board and clicking an already occupied cell.
6. The computer must also never overwrite an occupied cell.

7. Make the minimum necessary changes and preserve the existing project architecture.

Tell me which files need to be modified and provide the complete updated code for those files. :::

### One important thing

Since you already implemented the occupied-cell check during Task 1, **don't artificially change lots of files just to make Task 3 look different**. Your actual code already contains part of the solution. Task 3 is about ensuring that behavior is properly handled and tested.

After testing, your Git history can naturally look like:

TEXT

```
T a s k   1   !'   w i n / d r a w / g a m e - o v e r   f i x e s
T a s k   2   !'   s c o r e b o a r d   +   r o u n d   r e s e t
T a s k   3   !'   i n v a l i d / o c c u p i e d   m o v e   v a l i d a t i o n
```

That gives you a very clean progression to demonstrate in your lab submission.

USER

i am done with task3, now its time to do task 4 , the changes of task1, 2, 3 must remain intact, for task 4 "Choose who starts the round — X or O and Separate Restart Round and Reset Match controls" also explain the changes in simple terms

ASSISTANT

Yes. For **Task 4**, we'll build on Tasks 1–3 without removing any of their functionality.

The two new features are:

1. **Choose who starts the round — X or O**
2. **Separate controls:**
  - **Restart Round** `!` clears only the current board/round state; scoreboard stays.
  - **Reset Match** `!` clears the board **and** the entire scoreboard.

I recommend keeping the controls keyboard-based for now because your existing project already uses keyboard input and this avoids unnecessary changes to the Pygame layout. We can use:

- `R` = Restart Round
- `M` = Reset Match
- `X` = X starts next round
- `O` = O starts next round

This is a clean interpretation of the requirement and keeps the changes manageable.

## Files we need to change

We'll modify:

TEXT

```
game/game_engine.py
game/renderer.py
```

We do not need to change:

TEXT

```
game/rules.py
game/ai.py
main.py
```

Your Task 1, 2 and 3 logic remains intact.

## 1. game/game\_engine.py

Replace your current version with this:

PYTHON

```
"""
GameEngine: owns the board, turn state, round-end logic,
match scoreboard, and starting-player selection.

The human always plays X and the computer always plays O.
The selected starting player determines who makes the first move
of each round.
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'

class GameEngine:
    def __init__(self):
        # Round state
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = HUMAN_SYMBOL
        self.round_over = False
        self.winner = None

        # Starting player for the current round
        self.starting_player = HUMAN_SYMBOL

        # Match scoreboard
        self.x_wins = 0
        self.o_wins = 0
        self.draws = 0

    def handle_click(self, pos):
        # Do not accept moves after the round is over
        if self.round_over:
            return

        # Only X can make a mouse click
        if self.current_player != HUMAN_SYMBOL:
            return

        cell = board_pos_to_cell(pos)

        # Invalid click outside the board
        if cell is None:
            return
```

```

        row, col = cell

        # Occupied cells cannot be overwritten
        if self.board[row][col] is not None:
            return

        # Place X
        self.board[row][col] = HUMAN_SYMBOL

        # Check whether X ended the round
        self.check_round_end()

        if self.round_over:
            return

        # Give the turn to O
        self.current_player = COMPUTER_SYMBOL

        # Computer makes its move
        self._maybe_take_computer_turn()

def _maybe_take_computer_turn(self):
    # Do not let O move when it shouldn't
    if self.round_over or self.current_player != COMPUTER_SYMBOL:
        return

    move = choose_move(self.board)

    if move is None:
        return

    row, col = move

    # Extra safety: never overwrite an occupied cell
    if self.board[row][col] is not None:
        return

    # Place O
    self.board[row][col] = COMPUTER_SYMBOL

    # Check whether O ended the round
    self.check_round_end()

    if self.round_over:
        return

    # Give turn back to X
    self.current_player = HUMAN_SYMBOL

def handle_keydown(self, key):
    import pygame

    # R = Restart current round
    if key == pygame.K_r:
        self.restart_round()

    # M = Reset entire match
    elif key == pygame.K_m:
        self.reset_match()

    # X = Choose X to start
    elif key == pygame.K_x:
        self.starting_player = HUMAN_SYMBOL
        self.restart_round()

    # O = Choose O to start

```

```

        elif key == pygame.K_o:
            self.starting_player = COMPUTER_SYMBOL
            self.restart_round()

def restart_round(self):
    """
    Restart only the current round.

    The scoreboard remains unchanged.
    """

    self.board = [[None] * 3 for _ in range(3)]
    self.round_over = False
    self.winner = None

    # The selected starting player becomes
    # the first player of the new round.
    self.current_player = self.starting_player

    # If 0 starts, let the computer move automatically.
    self._maybe_take_computer_turn()

def reset_match(self):
    """
    Reset the entire match.

    The board, round state, and scoreboard are all cleared.
    """

    self.x_wins = 0
    self.o_wins = 0
    self.draws = 0

    # Start a fresh round while preserving
    # the currently selected starting player.
    self.restart_round()

def check_round_end(self):
    # Check winner FIRST.
    # This preserves Task 1's final-move win fix.

    winner = check_winner(self.board)

    if winner:
        self.round_over = True
        self.winner = winner

        if winner == HUMAN_SYMBOL:
            self.x_wins += 1
        elif winner == COMPUTER_SYMBOL:
            self.o_wins += 1

        return

    # If nobody won and the board is full, it is a draw.
    if is_board_full(self.board):
        self.round_over = True
        self.winner = None
        self.draws += 1

def draw(self, surface, font):
    from game import renderer

    renderer.draw_board(surface, self.board)

    # Scoreboard

```

```

    renderer.draw_scoreboard(
        surface,
        font,
        self.x_wins,
        self.o_wins,
        self.draws
    )

    # Current turn
    turn_label = (
        "Your turn (X)"
        if self.current_player == HUMAN_SYMBOL
        else "Computer's turn (O)"
    )

    renderer.draw_text(
        surface,
        font,
        turn_label,
        (10, 55)
    )

    # Starting player
    renderer.draw_text(
        surface,
        font,
        f"Starts: {self.starting_player}",
        (10, 80)
    )

    # Round result
    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"

        renderer.draw_banner(
            surface,
            font,
            f"{text} R=Restart Round | M=Reset Match"
        )

```

## 2. game/renderer.py

Your Task 2/3 `renderer.py` can mostly remain unchanged.

The only thing I recommend adding is a separate function for displaying the controls. This keeps rendering responsibilities inside `renderer.py`.

Use this complete version:

```

PYTHON
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 400, 500
BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100
WINDOW_SIZE = (WIDTH, HEIGHT)

```

```

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)

def board_pos_to_cell(pos):
    x, y = pos
    y -= BOARD_TOP

    if not (0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE):
        return None

    col = x // CELL_SIZE
    row = y // CELL_SIZE

    return int(row), int(col)

def draw_board(surface, board):
    surface.fill(COLOR_BG)

    for i in range(1, 3):
        pygame.draw.line(
            surface,
            COLOR_LINE,
            (i * CELL_SIZE, BOARD_TOP),
            (i * CELL_SIZE, BOARD_TOP + BOARD_SIZE),
            3
        )

        pygame.draw.line(
            surface,
            COLOR_LINE,
            (0, BOARD_TOP + i * CELL_SIZE),
            (BOARD_SIZE, BOARD_TOP + i * CELL_SIZE),
            3
        )

    for r in range(3):
        for c in range(3):
            symbol = board[r][c]

            if symbol is None:
                continue

            center = (
                c * CELL_SIZE + CELL_SIZE // 2,
                BOARD_TOP + r * CELL_SIZE + CELL_SIZE // 2
            )

            if symbol == 'X':
                offset = CELL_SIZE // 3

                pygame.draw.line(
                    surface,
                    COLOR_X,
                    (center[0] - offset, center[1] - offset),
                    (center[0] + offset, center[1] + offset),
                    6
                )

                pygame.draw.line(
                    surface,

```

```

        COLOR_X,
        (center[0] + offset, center[1] - offset),
        (center[0] - offset, center[1] + offset),
        6
    )

    else:
        pygame.draw.circle(
            surface,
            COLOR_O,
            center,
            CELL_SIZE // 3,
            6
        )

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_scoreboard(surface, font, x_wins, o_wins, draws):
    text = f"X Wins: {x_wins}    O Wins: {o_wins}    Draws: {draws}"
    draw_text(surface, font, text, (10, 20))

def draw_controls(surface, font):
    text = "X/O = Choose Starter    R = Restart    M = Reset Match"
    draw_text(surface, font, text, (10, 75))

def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))

    rect = surf.get_rect(
        center=(
            surface.get_width() // 2,
            BOARD_TOP + BOARD_SIZE + 40
        )
    )

    surface.blit(surf, rect)

```

## One small improvement to `GameEngine.draw()`

Since we created `draw_controls()`, change this:

```

PYTHON
renderer.draw_text(
    surface,
    font,
    f"Starts: {self.starting_player}",
    (10, 80)
)

```

to:

```

PYTHON
renderer.draw_controls(surface, font)

```

So your `draw()` section becomes:

PYTHON

```
def draw(self, surface, font):
    from game import renderer

    renderer.draw_board(surface, self.board)

    renderer.draw_scoreboard(
        surface,
        font,
        self.x_wins,
        self.o_wins,
        self.draws
    )

    turn_label = (
        "Your turn (X)"
        if self.current_player == HUMAN_SYMBOL
        else "Computer's turn (O)"
    )

    renderer.draw_text(
        surface,
        font,
        turn_label,
        (10, 55)
    )

    renderer.draw_controls(surface, font)

    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"

        renderer.draw_banner(
            surface,
            font,
            f"{text} R=Restart Round | M=Reset Match"
        )
```

## Simple explanation of Task 4

There are **three important concepts** we're adding.

### 1. Choosing who starts

Previously, every round automatically started with:

PYTHON

```
self.current_player = HUMAN_SYMBOL
```

So X always started.

Now we have:

PYTHON

```
self.starting_player = HUMAN_SYMBOL
```

This stores who should start the round.

Press:

TEXT

X

!' X starts.

Press:

TEXT

O

!' O starts.

If O starts, the computer automatically makes its first move through:

PYTHON

```
self._maybe_take_computer_turn()
```

## 2. Restart Round vs Reset Match

This is the biggest change in Task 4.

Previously we only had one reset:

TEXT

R

Now we have two different operations.

### R — Restart Round

It does:

TEXT

```
Clear board
Reset winner
Reset round-over state
Use selected starter
KEEP SCORE
```

Example:

TEXT

Before:

```
X Wins: 3
O Wins: 2
Draws: 1
```

Press R

After:

```
X Wins: 3
O Wins: 2
Draws: 1
```

Empty board

---

## M — Reset Match

It does:

```
TEXT
Clear X wins
Clear O wins
Clear draws
Clear board
Start fresh round
```

Example:

```
TEXT
Before:

X Wins: 3
O Wins: 2
Draws: 1

Press M

After:

X Wins: 0
O Wins: 0
Draws: 0

Empty board
```

Notice that **Reset Match does not reset the selected starter**. If you selected O as the starter, O remains the selected starter.

---

## Why we need separate methods

Instead of having everything inside `handle_keydown()`, we now have:

```
PYTHON
restart_round()
```

and:

```
PYTHON
reset_match()
```

This makes the difference very clear.

```
TEXT
restart_round()
! "
board + round state reset
! "
score stays
```

```
reset_match()
    !""
score reset
    !""
restart_round()
```

That's a clean design and makes it much easier to add more features later.

---

## Task 1, 2 and 3 are preserved

### Task 1 '

Still has:

- 8 winning lines
- Both diagonals
- Winner checked before draw
- No moves after game ends

### Task 2 '

Still has:

- X wins
- O wins
- Draws
- Visible scoreboard
- Round restart doesn't erase score

### Task 3 '

Still has:

- Occupied cells cannot be overwritten
- Invalid clicks don't modify the board
- Invalid clicks don't change the turn
- Valid moves work normally

### Task 4 Ø<Ý•

Adds:

- X/O starter selection
- **R** = Restart Round
- **M** = Reset Match
- Separate round and match reset logic

---

## One thing I would change from the code above

There is a **UI-space issue** with your current `400 × 500` window: the board already occupies `y=100` to `460`, leaving only 40 pixels underneath. So putting a long control message in the existing 75px area isn't ideal.

For a clean lab implementation, I recommend **not adding a visual controls line** and instead simply display:

TEXT

Starts: X

or

TEXT

Starts: O

at the top, while documenting the keyboard controls in your README/comments.

That avoids changing the window size and potentially affecting your existing click coordinates.

So I'd use:

PYTHON

```
renderer.draw_text(  
    surface,  
    font,  
    f"Starts: {self.starting_player}",  
    (10, 80)  
)
```

and **do not use** ``draw_controls()``.

The actual controls are:

TEXT

```
X !'  X  s t a r t s   n e x t   r o u n d  
O !'  O  s t a r t s   n e x t   r o u n d  
R !'  R e s t a r t   R o u n d  
M !'  R e s e t     M a t c h
```

---

## Testing checklist before you commit

---

### Test 1 — X starts

Press:

TEXT

X

Then restart with:

TEXT

R

X should get the first move when you click.

---

### Test 2 — O starts

Press:

TEXT

O

O should immediately make a random move.

You should see:

TEXT

Starts: 0

and an O on the board.

---

### Test 3 — Restart Round

Get a score such as:

TEXT

X Wins: 2      O Wins: 1      Draws: 0

Press:

TEXT

R

Expected:

TEXT

X Wins: 2      O Wins: 1      Draws: 0

but the board is empty.

---

### Test 4 — Reset Match

With a score such as:

TEXT

X Wins: 2      O Wins: 1      Draws: 1

press:

TEXT

M

Expected:

TEXT

X Wins: 0      O Wins: 0      Draws: 0

and a fresh board.

## Test 5 — Task 3 still works

After selecting O as starter, try clicking an occupied cell.

It must **not overwrite O**.

Try clicking outside the board.

It must **not change the turn or board**.

---

## Test 6 — Task 1 still works

Test both diagonals again after implementing Task 4. This is important because you want to make sure the new reset/starter logic didn't accidentally break your previous work.

---

## VibeCoding prompt for Task 4

Use this as your actual Task 4 prompt so your exported chat history clearly demonstrates what you asked the AI to do:

:::writing{variant="standard" id="82461" title="Task 4 Vibe Coding Prompt"} I have completed Tasks 1, 2 and 3 of my Python/Pygame Tic-Tac-Toelab. All existing functionality must remain intact.

Task 1 functionality that must remain:

- All 8 winning lines are detected, including both diagonals.
- A winning final move is counted as a win instead of a draw.
- No moves are accepted after a round ends.

Task 2 functionality that must remain:

- X wins, O wins and draws are tracked.
- The scoreboard is visible.
- Restarting a round does not clear the scoreboard.

Task 3 functionality that must remain:

- Occupied cells cannot be overwritten.
- Invalid clicks do not change the board.
- Invalid clicks do not change the turn.
- Valid moves continue working normally.

Now implement Task 4:

1. Allow the user to choose who starts each round: X or O.
2. X is still the human player and O is still the computer.
3. If O is selected as the starter, the computer should automatically make the first move.
4. Add a separate "Restart Round" control. It should clear only the current board and round state while preserving the scoreboard.
5. Add a separate "Reset Match" control. It should clear the scoreboard as well as the current round.
6. The selected starting player should remain selected when restarting a round.
7. Keep the existing random-move AI.
8. Do not remove or break any Task 1, Task 2 or Task 3 functionality.
9. Make the minimum necessary changes to the existing architecture.
10. Explain which files need to change and provide the complete updated code.

Use the existing Pygame interface and keyboard controls if possible rather than redesigning the entire UI. :::

**One important note:** because your assignment says "choose who starts the round," this implementation lets you press **X/O before starting/restarting a round**. It doesn't introduce a separate graphical menu/button system, which keeps the change small and preserves your existing game.

After you test this, **commit Task 4 separately** so your Git history clearly shows Tasks 1 ! 2 ! 3 ! 4.

USER

the final text is o/p is having scaling problem in which file should i do the changes

ASSISTANT

If by **scaling problem** you mean the final text/banner like "**X wins! R=Restart Round | M=Reset Match**" is getting cut off, overlapping, or going outside the window, the main change should be in ``game/renderer.py``.

The problem is that this text is too long for your current **400 px wide** window.

**Best fix: change `draw_banner()` in `game/renderer.py`**

Replace your current `draw_banner()` with:

PYTHON

```
def draw_banner(surface, font, text):
    # Use a smaller font for long messages
    banner_font = pygame.font.SysFont("consolas", 16)

    surf = banner_font.render(text, True, (180, 40, 40))

    rect = surf.get_rect(
        center=(
            surface.get_width() // 2,
            BOARD_TOP + BOARD_SIZE + 20
        )
    )

    surface.blit(surf, rect)
```

Then in ``game_engine.py``, keep the banner text reasonably short:

PYTHON

```
if self.round_over:
    text = f"{self.winner} wins!" if self.winner else "Draw!"

    renderer.draw_banner(
        surface,
        font,
        f"{text} R: Restart | M: Reset Match"
    )
```

**Why this works**

Your window is:

PYTHON

```
WIDTH, HEIGHT = 400, 500
```

and your normal font is:

PYTHON

```
font = pygame.font.SysFont("consolas", 22)
```

A 22px font is too large for a long sentence inside a 400px-wide window.

So we're making **only the banner font smaller**:

PYTHON

```
banner_font = pygame.font.SysFont("consolas", 16)
```

You don't need to change your board size or the rest of the UI.

### If the problem is instead the scoreboard text

If you're talking about:

TEXT

```
X Wins: 10    O Wins: 10    Draws: 10
```

getting cut off, then the change should also be in `renderer.py`, specifically `draw_scoreboard()`.

Tell me **which text is scaling/cutting off** (scoreboard, `Starts: X`, or the final `X wins!` message), and I can give you the exact replacement code.

If you want, I can:

- Fix scaling of final round result banner text in `renderer.py`
- Adjust scoreboard text rendering in `renderer.py` to fix scaling issues
- Clarify which text has scaling problem to recommend precise fix